

Esposizione 14 luglio — divisione e discorsi

Documento di preparazione per l'esposizione di Ingegneria del Software. Contiene: chi presenta cosa, il discorso parola per parola (per ora quello di Emanuele, backend), la strategia per gestire le domande del prof e — soprattutto — la spiegazione test per test, così chi parla sa davvero cosa dice quando cita “101 test”.

Da leggere insieme a [presentazione-14-luglio.md](https://github.com/emanuele/14-luglio) (scaletta, demo, piano B, checklist).

Ordine delle slide e divisione

Le slide sono in ordine di ciclo di vita: prima la progettazione, poi l'architettura, poi gli strati. I blocchi di ciascuno sono consecutivi, così il microfono non rimbalza.

#	Slide	Chi
1	Copertina	Elio
2	Il problema	Elio
3	L'app	Elio
4	Requisiti funzionali	Elio
5	Requisiti non funzionali e priorità	Elio
6	Processo Scrum	Elio
7	Architettura pipeline a tre stadi	Andrea
8	Scraper	Andrea
9	Rischi e scraping etico	Andrea
10	Backend: API REST su FastAPI	Emanuele
11	Schema del database	Emanuele
12	Pattern di progettazione e architetturali	Emanuele
13	Testing e qualità	Emanuele
14	Decisioni di design difese	Emanuele
15	Frontend: una sola codebase	Yonas
16	Frontend: schermate e scelte tecniche	Yonas
17	Demo dal vivo	Yonas
18	Chiusura	Yonas

Strategia con le domande del prof

Non anticipare *tutto* ciò che può attirare una critica: se lo fai, il discorso si allunga e sembri insicuro. La regola è:

- **Preempti** (le dici tu, nel discorso) solo le due o tre domande che il prof fa quasi di sicuro, così gli togli il “gotcha”.

- **Tieni pronte** le altre risposte e le usi solo se te le chiede.

Owning un limite come scelta con trade-off (“lo sappiamo, abbiamo scelto X perché Y”) vale molto più che nascondere.

Per la parte backend, le tre da preempire sono: gli orari salvati come stringa JSON, il vincolo di unicità con l’anno NULL, e SQLite in produzione.

Discorso — Andrea (architettura e scraper, slide 7–9)

Circa tre-quattro minuti.

[Aggancio da Elio — slide 7, Architettura pipeline]

«Grazie Elio. Adesso vediamo com’è fatto il sistema. L’architettura è una pipeline a tre stadi: uno scraper in Python raccoglie i dati dai siti dei cinema e li scrive in file JSON; il backend li importa e li serve via API; l’app li consuma. La cosa importante sono i contratti espliciti tra gli stadi — file JSON tra scraper e backend, API REST tra backend e app — così ogni stadio è indipendente e si testa da solo. Io mi sono occupato del primo, lo scraper.»

[Slide 8 — Scraper]

«I tre cinema pubblicano i dati in modi completamente diversi, quindi ho usato un connettore per cinema: è il pattern Strategy, stessa interfaccia e una tecnica diversa per ognuno.

PostModernissimo è un sito Next.js, e ne leggo il payload interno più l’HTML. The Space ha un’API REST, e la interrogo direttamente. UCI ha un’API non documentata, che ho ricostruito guardando il traffico di rete. Una volta raccolti, per ogni film normalizzo il titolo, tolgo i duplicati, arricchisco i metadati da Wikidata, tengo traccia delle differenze rispetto alla notte prima, e scrivo tre file JSON. In produzione gira da solo ogni notte alle tre con un timer di sistema, ed è coperto da 75 test.»

[Slide 9 — Rischi e scraping etico. Qui anticipi le due domande vere.]

«Due cose le dico io, perché sono le domande che vi verrebbe da fare.

La prima: “e se un sito cambia e lo scraper si rompe?”. È il rischio numero uno, e l’abbiamo messo in conto. Le difese sono tre: un connettore per cinema, così se se ne rompe uno gli altri continuano; gli errori sono isolati e non bloccano l’intera raccolta; e il delta tracking tiene i dati della notte prima invece di cancellarli. Più i 75 test che fissano il comportamento di ogni parser.

La seconda, più delicata: “ma è lecito fare scraping?”. Sì, e con attenzione. Rispettiamo il robots.txt, limitiamo a una richiesta al secondo, giriamo di notte per non pesare sui server, ci identifichiamo con uno User-Agent che ha un contatto reale, e rimandiamo al sito del cinema per l’acquisto: portiamo traffico, non lo togliamo. È un’app gratuita e didattica, non commerciale.»

[Passaggio a Emanuele]

«Questi tre file JSON li passo al backend, e come diventano un database ve lo racconta Emanuele.»

Discorso — Emanuele (backend e database, slide 10–14)

Circa quattro-cinque minuti. Le indicazioni tra parentesi quadre sono di regia, non si leggono.

[Aggancio da Andrea — slide 10, Backend]

«Grazie Andrea. Lui vi ha mostrato *come* nascono i dati; io vi spiego come li conserviamo e come li serviamo all'app. Il backend è un'API REST in FastAPI, organizzata a livelli, ognuno con una responsabilità sola: i router ricevono l'HTTP e validano, i service hanno la logica, i repository sono le uniche classi che toccano il database, i model sono le tabelle. Si potrebbe dire che per tre cinema è troppa struttura — ma è proprio questa separazione che ci ha permesso di testare ogni pezzo in isolamento, ed è la buona pratica che il corso chiede di dimostrare. Sono undici endpoint documentati da Swagger; per l'app è tutto in sola lettura, e i due di amministrazione sono protetti da token.»

[Slide 11 — Schema del database. Qui anticipi i due punti caldi.]

«Tre tabelle: cinema, film, e gli spettacoli che li collegano in una certa data. Due scelte le anticipo io, perché sono le domande giuste da farsi.

Primo: gli orari di una proiezione li salvo come stringa JSON in un'unica colonna, non in una tabella separata. È una denormalizzazione consapevole: gli orari di un film, in un cinema, in un giorno si usano sempre insieme — non serve interrogarli uno per uno — così evito una quarta tabella e una join a ogni lettura. Se domani servisse filtrare per singolo orario, si normalizza.

Secondo: il vincolo di unicità sui film è su titolo normalizzato più anno. Quando l'anno è NULL, in SQL NULL è diverso da NULL, quindi lì il database da solo non deduplica: lo garantiamo nel codice, con un controllo prima dell'insert. È un limite noto, e lo dichiariamo.

Per il resto: chiavi esterne in cascade per l'integrità referenziale, e indici sulle colonne che interroghiamo di più, data e film.»

[Slide 12 — Pattern. È la slide che il prof ha già segnalato: qui giochi in anticipo.]

«Abbiamo usato più pattern, ma vanno distinti — e in questa slide l'abbiamo corretto. L'unico design pattern classico del Gang of Four è lo Strategy, nello scraper. Repository, Service Layer e DTO non sono GoF: sono pattern architetturali, quelli di Fowler, che organizzano l'architettura a livelli. La Dependency Injection è un principio, l'Inversion of Control, ed è ciò che nei test ci fa iniettare un database finto al posto di quello vero. La Factory è l'idioma con cui costruiamo l'app.»

[Slide 13 — Testing e qualità]

«Centouno test in tutto, ventisei sul backend, divisi in due tipi. Dieci sono test di unità sui repository: provano l'accesso ai dati isolato, senza HTTP — per esempio che l'upsert di un film lo inserisce e, se rilanciato, lo aggiorna senza duplicarlo, o che la ricerca ignora gli accenti. Dodici sono end-to-end sugli endpoint: fanno partire l'app HTTP vera con un database in memoria iniettato al posto di quello reale, e verificano le risposte — i 404 quando qualcosa non esiste, la validazione della ricerca a minimo due caratteri, e che l'endpoint di amministrazione

rifiuti chi non ha il token. Girano su GitHub Actions a ogni push. E i comportamenti che abbiamo corretto sono fissati da test di regressione, così i bug non tornano — compreso quello che vedete raccontato: “Odissea” mostrava un orario invece di dodici, perché il vincolo di unicità ne teneva uno solo. Diagnosticato, corretto unendo gli orari prima di scrivere.»

[Slide 14 — Decisioni di design difese. Qui anticipi SQLite.]

«Chiudo con due scelte che difendo. SQLite anche in produzione: tre cinema, un paio di centinaia di spettacoli, e l’app legge soltanto. Un database server aggiungerebbe amministrazione senza un beneficio misurabile. So che la domanda è “e la concorrenza?”: SQLite serializza le scritture, ma noi scriviamo solo col seed notturno mentre di giorno si legge — quindi non è un problema. E il repository isola la scelta: migrare a PostgreSQL cambia soprattutto la stringa di connessione, perché l’ORM astrae il dialetto; onestamente un paio di tipi di colonna li ritoccheremmo, ma il codice sopra no. Seconda scelta, Wikidata invece di TMDb: niente API key, niente limiti commerciali, licenza aperta.»

[Passaggio a Yonas]

«Con i dati serviti in modo affidabile, passo a Yonas che vi mostra l’app dal vivo.»

Approfondimento: cosa verificano davvero i 101 test

Serve per rispondere con cognizione se il prof chiede “e questi test cosa provano?”. Sono 101 raccolti da `pytest`: 26 sul backend, 75 sullo scraper.

Backend — 26 test (dominio di Emanuele)

Stanno in `backend/tests/`, divisi in due file. Il conteggio arriva a 26 perché alcuni test sono parametrizzati (una funzione, più casi).

Unità sui repository (`test_repositories.py`) — provano l’accesso ai dati in isolamento, senza HTTP:

- `test_upsert_insert_new_cinema` — inserisce un cinema nuovo.
- `test_upsert_updates_existing_cinema` — rilanciando l’upsert, aggiorna quello esistente invece di duplicarlo (idempotenza).
- `test_list_all_ordered_by_name` — la lista dei cinema torna ordinata per nome.
- `test_normalize_title` — la normalizzazione del titolo (via accenti e punteggiatura) funziona.
- `test_upsert_from_scraper_insert_and_lookup_by_natural_key` — inserisce un film e lo ritrova con la chiave naturale (titolo normalizzato + anno).
- `test_upsert_from_scraper_updates_only_non_null` — quando arricchisce un film, sovrascrive solo i campi non nulli, così un arricchimento povero non cancella dati già presenti (è la logica della decisione D1).
- `test_search_by_title_ignora_accenti` — la ricerca per titolo ignora gli

accenti.

- `test_list_in_programming` — la lista dei film in programmazione.
- `test_showing_upsert_and_joinedload` — inserisce uno spettacolo con più orari (es. `["19:30", "22:00"]`) e lo ricarica con film e cinema agganciati in un colpo solo (joinedload: niente query N+1). È il test più vicino al caso “Odissea”.
- `test_count_by_cinema_only_future` — conta gli spettacoli per cinema tenendo solo quelli futuri.

End-to-end sugli endpoint (`test_routers.py`) — fanno partire l’app HTTP con `TestClient` e un DB in memoria:

- `test_health_returns_ok` — l’endpoint di health risponde.
- `test_list_cinemas_empty/test_list_cinemas_with_data` — lista cinema, vuota e popolata.
- `test_get_cinema_by_slug_returns_with_count` — dettaglio cinema con il conteggio degli spettacoli.
- `test_get_cinema_not_found` — un cinema inesistente restituisce 404.
- `test_films_today_empty` — i film di oggi quando non ce ne sono.
- `test_get_film_detail_not_found/`
`test_get_film_detail_returns_detail_schema` — dettaglio film: 404 se non esiste, schema corretto se esiste.
- `test_search_requires_min_2_chars` — la ricerca rifiuta le query sotto i due caratteri.
- `test_search_returns_matching_films` — la ricerca restituisce i film che combaciano.
- `test_admin_reimport_requires_token/`
`test_admin_reimport_rejects_wrong_token` — l’endpoint di re-import rifiuta chi non ha il token o ne ha uno sbagliato.

Scraper — 75 test (dominio di Andrea, qui per contesto)

Stanno in `scraper/tests/`, per categoria:

- **Normalizzazione e deduplica** (`test_normalizer.py`, 12) — normalizzazione dei titoli e fuzzy matching, per capire che “Dune – Parte 2” e “DUNE Parte 2” sono lo stesso film.
- **Metadati** (`test_metadata.py`, 13) — l’arricchimento da Wikidata.
- **Parser dei connettori** — `test_thespace.py` (10), `test_uci.py` (4), `test_postmodernissimo.py` (3), `test_postmod_detail_reconcile.py` (2): ogni connettore letto contro fixture registrate.
- **Delta tracking** (`test_delta.py`, 7) — tenere i dati vecchi e marcare aggiunte, aggiornamenti e rimozioni tra una run e l’altra.
- **Regressioni** (`test_fix_validation.py`, 7) — “congelano” i bug già corretti così non tornano: UCI filtra le proiezioni per giorno, PostModernissimo scarta gli stub-evento e i duplicati, The Space filtra per data, la deduplica fa l’unescape dell’HTML prima di confrontare.

- **Rete** (`test_http.py`, 7) — retry e rate limiting.
- **Configurazione e modelli** (`test_config.py`, 6; `test_models.py`, 4).

Domande scomode e risposte (Q&A)

Backend (Emanuele)

Da dire nel discorso (preempt):

- **Orari come stringa JSON (viola la 1NF?)** — denormalizzazione consapevole: gli orari di una proiezione si usano sempre insieme, non serve interrogarli singolarmente; si evita una tabella e una join. Normalizzabile in futuro.
- **UNIQUE(titolo, anno) con anno NULL** — in SQL `NULL ≠ NULL`, quindi lì la deduplica non la fa il DB ma il codice, con un controllo prima dell'insert.
- **SQLite in produzione** — carico minimo, sola lettura dall'app, un solo writer notturno; le scritture serializzate non sono un problema. Il repository isola la scelta.

Da tenere pronte (rispondi solo se chiede):

- **Slug PK cinema vs int PK film** — i cinema sono tre, stabili, con nomi unici: slug leggibile. I film sono centinaia con titoli fragili (apostrofi, remake omonimi): chiave artificiale più UNIQUE (decisione D3).
- **Migrare a PostgreSQL è solo la connection string?** — l'ORM astrae il dialetto, quindi in gran parte sì; onestamente un paio di tipi di colonna andrebbero ritoccati.
- **Copertura dei test / test banali?** — testiamo il comportamento, non le righe: parser con fixture registrate, endpoint con DB in memoria, più i test di regressione sui bug reali.
- **Perché FastAPI?** — validazione automatica con Pydantic, Swagger generato, async nativo.
- **Auth solo con token sugli admin?** — proporzionato a due endpoint interni; JWT/OAuth completo sarebbe over-engineering per un pannello di re-import.

Scraper (Andrea)

Da dire nel discorso (preempt):

- **È lecito fare scraping?** — robots.txt rispettato, rate limit una richiesta al secondo, run notturna, User-Agent con contatto reale, link al sito del cinema per l'acquisto. App gratuita e didattica, non commerciale.
- **E se un sito cambia?** — un connettore per cinema (Strategy), errori isolati, delta tracking che tiene i dati vecchi, 75 test sui parser.

Da tenere pronte:

- **Cos'è il fallback CloakBrowser?** — un browser headless usato solo se l'API di The Space cambia o smette di rispondere; legge la stessa pagina pubblica che vedrebbe un utente, sempre col rate limit. Non aggira login né paywall: i palinsesti sono pubblici.
- **API non documentata di UCI** — è l'API che il sito stesso usa per popolarsi; leggiamo solo dati pubblici (orari), rispettando i limiti. Se cambia, si rompe solo quel connettore.

- **Siti in JavaScript non scrapabili (R-03)?** — PostModernissimo è Next.js con contenuto lato server, quindi leggibile; per i casi peggiori resta il fallback browser.
- **Wikidata invece di TMDB** — niente API key, niente limiti commerciali, licenza aperta (decisione D1).

Da fare

- Discorsi di Elio (apertura e analisi) e Yonas (frontend, demo, chiusura): li scrivono loro, stesso taglio.